

USER

userId : 사용자 ID
Password : 사용자 비밀번호
userName : 사용자 이름
publicKey : 인증용 공개키
encryptedPrivateKey : 인증용 개인키
chatPublicKey : 채팅용 공개키
encryptedChatPrivateKey : 채팅용 개인키

User Repository

userId, userName, publicKey,
chatPublicKey : 그대로 저장
Password : 해시값으로 저장
encryptedPrivateKey,
encryptedChatPrivateKey : 암호문으로 저장

**Users.json
저장**

CHAT

encryptedMessage : 문자
senderId : 보내는 사람 ID
receiverId : 받는 사람 ID

Chat Repository

senderId, receiverId : 그대로 저장
encryptedMessage : 암호문으로 저장

**message.json
저장**

HashUtil

Hash Algorithm : SHA256
Hash값에 대해서 1000번 반복 계산
- 회원가입 시 생성되는 비밀번호에 대해
해시값 생성

CipherUtil

Algorithm : AES - GCM 사용
IV : 12
SEED : 16
키 길이 32로
Encrypt(평문, 키) : 평문에 대해서 키로 암호화
- SEED값과 평문을 합쳐 암호화
Decrypt(암호문, 키) : 암호문에 대해서 키로 복호화

ChatKeyPair

EC Algorithm : EC
generateKeyPair() : 키 쌍 생성
sharedSecret(개인키, 공개키) : 채팅
용 공유키 계산

DigitalSign

generateChallenge(길이) : challenge값 생성
generateSignature(개인키, challenge)
: 전자서명값 생성
verifySignature(공개키, challenge, 서명값)
: 전자서명 검증

UserService

currentUser : 로그인 확인용

회원가입 joinMember()

- 회원 ID 입력
- 비밀번호 입력
 - > 비밀번호에 대해서 해시값 생성
- 로그인용 키 쌍 생성
 - > 개인키는 비밀번호 해시값으로 암호화
 - > 공개키는 그대로 저장
- 채팅용 키 쌍 생성
 - > 개인키는 비밀번호 해시값으로 암호화
 - > 공개키는 그대로 저장

로그인 login()

- 회원 ID 입력
 - > 회원이 존재하는 회원인지 확인
- 비밀번호 입력
 - > 개인키 복호화
 - > 개인키로 전자서명값 생성
 - > 공개키로 전자서명 검증

ChatService

메시지 보내기

sendChat(보내는 사람 ID, 받는 사람 ID, 비밀번호)

- 보내는 사람/받는 사람 ID의 매개변수로 user repository에 존재하는지 확인
- 채팅용 송신자의 개인키 + 채팅용 수신자의 공개키로 공유키 생성
- 메시지 입력

메시지 받기

getChat(받는 사람 ID, 보내는 사람 ID, 비밀번호)

- 받는 사람/보내는 사람 ID의 매개변수로 user repository에 존재하는지 확인
- 채팅용 수신자의 개인키 + 채팅용 송신자의 공개키로 공유키 생성
- 메시지 읽기